Lecture 5

Key Machine Learning Concepts - Explained wth Linear Regression
Academic Year 1447 Term 1
Dr. Jomana Bashatah
Slides adapted from Casten Lange

Loading Required Libraries

```
library(tidymodels)
library(rio)
library(kableExtra)
library(janitor)
library(shiny)
DataMockup =
import("https://raw.githubusercontent.com/
jomanab-cloud/ISE351-25-
Presentations/afba6ed1275d81610b8b4214e765
fac73046b2cb/DataStudyTimeMockup.rds")
```

Learning Objectives

- Reviewing the basic idea behind linear regression
- Learning how how to measure predictive quality with Mean Square Error (MSE).
- Understanding the role of parameters in a machine learning model in general and in linear regression in particular
- Calculating optimal regression parameters using OLS
- Finding optimal regression parameters by trial and error
- Distinguish between unfitted and fitted models
- Using the `tidymodels` package to split observations from a dataset randomly into a training and testing dataset.
- Understanding how categorical data such as the gender of a person (female/male) can be transformed into numerical dummy variable.
- Being able to distinguish between dummy encoding and one-hot encoding
- Using `tidymodels` including model design and data pre-processing (recipes) to analyze housing prices.

Jumping Right Into It

Univariate OLS With a Real World Example

Data Description

- King County House Sale dataset (Kaggle 2015). House sales prices from May 2014 to May 2015 for King County in Washington State.
- Several predictor variables. For now we use only $Sqft$
- We will only use 100 randomly chosen observations from the total of 21,613 observations.
- We only use Sqft as predictor variable for now.

Loading the Data and Assigning Training and Testing Data (Manually)

```
DataHouses=

import("https://raw.githubusercontent.com/jomanab-cloud/ISE351-25-Presentations/80f7609c6bcb4887517dcaff62908459cee33e33/HousingData.csv") |>
  clean_names("upper_camel") |>
  select(Price, Sqft=SqftLiving)

# Manually generating DataTrain and DataTest
set.seed(7771)
DataTrain= sample_n(DataHouses, 100)
DataTest= sample_n(DataHouses, 50)
head(DataTrain)
```

Price	Sqft
517000	1180
236000	1300
490000	2800
129000	1150
257000	1400
312500	870

How Much is a House Worth in King County

A house with average properties should be predicted with an average price!

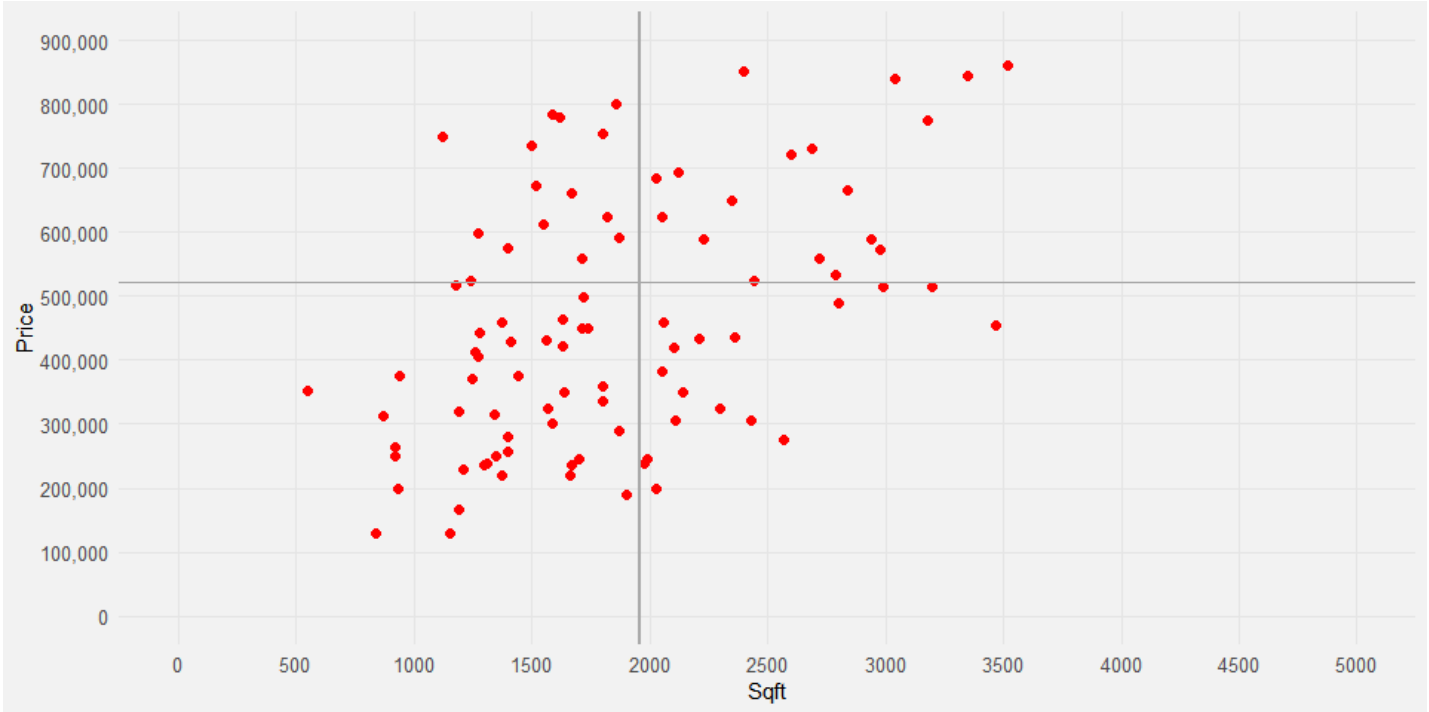
```
MeanSqft=mean(DataTrain$Sqft)
cat("The mean square footage of a house in King
county is:", MeanSqft)
```

```
The mean square footage of a house in King county
is: 1956.7
```

```
MeanPrice=mean(DataTrain$Price)
cat("The mean price of a house in King county
is:", MeanPrice)
```

```
The mean price of a house in King county is:
521294.2
```

Predicting the Price of an Average Sized House as the Average of all House Prices



From Unfitted to Fitted Model

How Does the Unfitted Model Looks Like

$$\underset{\breve{y}}{\widehat{Price}} = \underset{\breve{m}}{\beta_1} \underset{\breve{x}}{Sqft} + \underset{\breve{b}}{\beta_0}$$

Fitting the Model with Tidymodels

```
RecipeHouses= recipe(Price~Sqft,
data=DataTrain)

ModelDesignOLS= linear_reg() %>%
                set_engine("lm")
%>%

set_mode("regression")

WFModelHouses = workflow() %>%
  add_recipe(RecipeHouses) %>%
  add_model(ModelDesignOLS) %>%
  fit(DataTrain)

tidy(WFModelHouses)
```

term	estimate	std.error	statistic	p.value
(Intercept)	52509.1036	64183.47011	0.8181095	0.4152795
Sqft	239.5794	30.57481	7.8358445	0.0000000

Unfitted Model VS Fitted Workflow Model

Unfitted Model:

$$\underset{\breve{\hat{y}}}{\widehat{Price}} = \underset{\breve{m}}{\beta_1} \underset{\breve{x}}{Sqft} + \underset{\breve{b}}{\beta_0}$$

```
tidy(WFModelHouses)
```

term	estimate	std.error	statistic	p.value
(Intercept)	52509.1036	64183.47011	0.8181095	0.4152795
Sqft	239.5794	30.57481	7.8358445	0.0000000

Fitted Model:

$$\underset{\hat{y}}{\widehat{Price}} = \underset{m}{240}\underset{x}{Sqft} + \underset{b}{52509}$$

Interpretation and Significance

term	estimate	std.error	statistic	p.value
(Intercept)	52509.1036	64183.47011	0.8181095	0.4152795
Sqft	239.5794	30.57481	7.8358445	0.0000000

$$\widehat{Price} = 240 \cdot Sqft + 52509$$
$$(+240) \quad = 240 \cdot (+1) + (+0)$$
$$(+480) \quad = 240 \cdot (+2) + (+0)$$
$$(+720) \quad = 240 \cdot (+3) + (+0)$$

For each extra $Sqft$ the predicted price increases by $240

The variable $Sqft$ is significant. I.e., the probability that the related coefficient β_1 equals zero is extremely small.

Evaluating Predictive Quality with the Testing Dataset

```
DataTestWithPred=augment(WFModelHou
ses, new_data=DataTest)
metrics(DataTestWithPred,
truth=Price, estimate=.pred)
```

.metric	.estimator	.estimate
rmse	standard	1.634755e+05
rsq	standard	6.256008e-01
mae	standard	1.320503e+05

Univariate Linear Regression - Data Table and Goal

The Regression:

$$\hat{y}_i = \beta_1 x_i + \beta_2$$

The Goal

Find values for β_1 and β_2 that minimize the prediction errors $(y_i - \hat{y}_i)^2$

The Data Table

Mockup Training Dataset

i	yGrade	xStudyTime
1	65	2
2	82	3
3	93	7
4	93	8
5	83	4

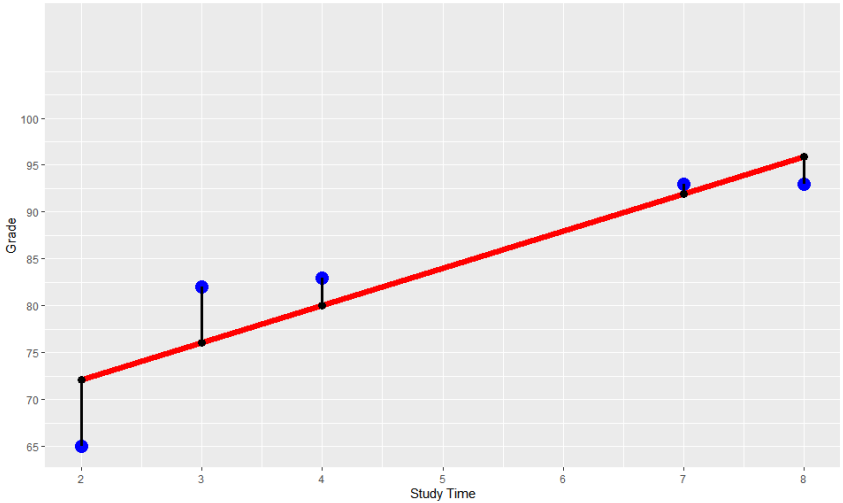
Univariate Linear Regression - Data Diagram and Goal

The Regression:

$$\hat{y}_i = \beta_1 x_i + \beta_2$$

The Goal

Find values for β_1 and β_2 that minimize the prediction errors $(y_i - \hat{y}_i)^2$



How to Measure Prediction Quality

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

$$\Leftrightarrow$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N}\left(\overbrace{\beta_1 x_i + \beta_2}^{\text{Prediction } i} - y_i\right)^2$$

$$\text{Error}_i$$

Note: when the data are given (i.e., x_i and y_i are given), the MSE depends only on the choice of β_1 and β_2

How to Measure Prediction Quality with the MSE

$$MSE = \frac{(\beta_1 x_1 + \beta_2 - y_1)^2 + (\beta_1 x_2 + \beta_2 - y_2)^2 + \cdots + (\beta_1 x_5 + \beta_2 - y_5)^2}{5}$$

$$\Leftrightarrow$$

$$MSE = \frac{1}{5}\left[\underbrace{\left(\overbrace{\beta_1 \cdot 2 + \beta_2}^{\text{Prediction 1}} - 65\right)^2}_{\text{Error 1}} + \underbrace{\left(\overbrace{\beta_1 \cdot 3 + \beta_2}^{\text{Prediction 2}} - 82\right)^2}_{\text{Error 2}} + \underbrace{\left(\overbrace{\beta_1 \cdot 7 + \beta_2}^{\text{Prediction 3}} - 93\right)^2}_{\text{Error 3}} + \underbrace{\left(\overbrace{\beta_1 \cdot 8 + \beta_2}^{\text{Prediction 4}} - 93\right)^2}_{\text{Error 4}} + \underbrace{\left(\overbrace{\beta_1 \cdot 4 + \beta_2}^{\text{Prediction 5}} - 83\right)^2}_{\text{Error 5}}\right]$$

Custom R Function to Calculate MSE

Function Call:

```
library(rio)
FctMSE = import("https://raw.githubusercontent.com/jomanab-cloud/ISE351-25-
Presentations/ae02842f37d063b3a1b7f70f57bee0ade556e482/FctMSE.rds")
VecBetaTest=c(4,61)
ResultMSE=FctMSE(VecBetaTest, DataMockup)
print(ResultMSE)
```

```
[1] 29.8
```

Function Definition:

```
FctMSE=function(VecBeta, data){
                          Beta1=VecBeta[1]
                          Beta2=VecBeta[2]
                          data=data |>
                          rename(Y=1, X=2) |>
                          mutate(YPred=Beta1*X+Beta2) |>
                          mutate(Error=YPred-Y) |>
                          mutate(ErrorSqt=Error^2)

                          MSE=mean(data$ErrorSqt)

                          return(MSE)}
```

How to Find Optimal Values for β_1 AND β_2

Method 1:

Calculate optimal values for the parameters (the βs) based on Ordinary Least Squares (OLS) using two formulas (**Note**, this method works only for linear regression)

Method 2:

We can use a **systematic trial and error process**.

Method 1: Calculate Optimal Parameters (Only for OLS)

$$\beta_{1,opt} = \frac{N\sum_{i=1}^{N} y_i x_i - \sum_{i=1}^{N} y_i \sum_{i=1}^{N} x_i}{N\sum_{i=1}^{N} x_i^2 - \left(\sum_{i=1}^{N} x_i\right)^2} = 3.96$$

$$\beta_{2,opt} = \frac{\sum_{i=1}^{N} y_i - \beta_1 \sum_{i=1}^{N} x_i}{N} = 64.18$$

```
DataTable=DataMockup |>

mutate(GradeXStudyTime=Grade*StudyTime) |>

mutate(StudyTimeSquared=StudyTime^2)


kable(DataTable |> mutate(i=1:5) |>
select(i,everything()),
      caption="Mockup Training Dataset",
      align = "c")
```

i	Grade	StudyTime	GradeXStudyTime	StudyTime Squared
1	65	2	130	4
2	82	3	246	9
3	93	7	651	49
4	93	8	744	64
5	83	4	332	16

Mockup Training Dataset

Grade	StudyTime	GradeXStudyTime	StudyTimeSquared
416	24	2103	142

Column Sums

Finding Optimal Values in R

1. Define and save the recipe that indicates which variable is the outcome and which is the predictor variable, together with the data argument.

Note that we do not add any step_ commands for data pre-processing since we can use the mockup data as is. 2. Define the model design in three steps that determine:

- The nameofthemachinelearningmodel(linear_reg()).
- The R packageusedforthatmodel(lm).
- The mode used—which is either “classification”or“regression”.

3. Add the recipe and the model design to the workflow and fit the workflow to the data.

```
library(tidymodels)
RecipeMockup=recipe(Grade~StudyTime, data=DataMockup)
ModelDesignLinRegr=linear_reg() |>
set_engine("lm") |>
set_mode("regression")
WFModelMockup=workflow() |>
add_recipe(RecipeMockup) |>
add_model(ModelDesignLinRegr) |>
fit(DataMockup)
```

Method 2: Use a Systematic Trial and Error Process

- Grid Search (aka Brute Force):

1. For a given range of β_1 and β_2 values, build a table with pairs of all combinations of these β_s
2. Then use our custom `FctMSE()` command to calculate a *MSE* for each β pair.
3. Find the β pair with the lowest *MSE*

- **Optimizer**: Use the R build-in optimizer. Push the start values for β_1 and β_2 together with the data to the optimizer as arguments. The rest is done by the optimizer.

See the R script in the footnote to see both algorithms in action.

MULTIVARIATE OLS WITH A REAL WORLD DATASET

Multivariate OLS with a Real World Dataset

Data

```
library(rio)
DataHousing =

import("https://raw.githubusercontent.com/jomanab-cloud/ISE351-25-Presentations/f1259ac81b553bd18a1478f1f830c4e5e70647a9/HousingDataSmall.csv")
```

- King County House Sale dataset (Kaggle 2015). House sales prices from May 2014 to May 2015 for King County in Washington State.
- Several predictor variables.
- We will use all 21,613 observations.

Multivariate Analysis — Three Predictor Variables

Sqft: Living square footage of the house

Grade: Indicates the condition of houses (1 (worst) to 13 (best))

Waterfront: Is house located at the waterfront (*yes* or *no*)

```
library(tidyverse); library(rio); library(janitor);
library(tidymodels)
DataHousing =
  import("https://raw.githubusercontent.com/jomanab-cloud/ISE351-
25-
Presentations/80f7609c6bcb4887517dcaff62908459cee33e33/HousingData.
csv") |>
  clean_names("upper_camel") |>
  select(Price, Sqft=SqftLiving, Grade, Waterfront)
```

Unfitted Model

$$Price = \beta_1 Sqft + \beta_2 Grade + \beta_3 Waterfront_{yes} + \beta_4$$

Multivariate Real World Dataset — Splitting

```
set.seed(777)
Split7030=initial_split(0.7,data=DataHousing, strata = Price, breaks = 5)
DataTrain=training(Split7030)
DataTest=testing(Split7030)
```

DataTrain

Price	Sqft	Grade	Waterfront
221900	1180	7	no
180000	770	6	no
189000	1200	7	no
230000	1250	7	no
252700	1070	7	no
240000	1220	7	no

DataTest

Price	Sqft	Grade	Waterfront
1230000	5420	11	no
257500	1715	7	no
291850	1060	7	no
229500	1780	7	no
530000	1810	7	no
650000	2950	9	no

Dummy and One-Hot Encoding

One-Hot Encoding

```
OneHotTable=tibble(Waterfront_yes=c(0,0,1,0),Waterfront_no=c(1,1,0,
1))
print(OneHotTable)
# A tibble: 4 × 2
  Waterfront_yes Waterfront_no
           <dbl>         <dbl>
1              0             1
2              0             1
3              1             0
4              0             1
```

One-hot encoding is easier to interpret but causes problems in OLS (dummy trap) because one variable is redundant. We can calculate one variable from the other (*perfect multicollinearity*):

$$Waterfront_{yes} = 1 - Waterfront_{no}$$

Dummy and One-Hot Encoding

Dummy Coding

We use one variable less than we have categories. Waterfront has two categories. Therefore, we use one variable (e.g., Waterfront_yes):

Dummy Encoding Example

```
DummyTable=tibble(Waterfront_yes=c(0,0,1,0))
print(DummyTable)
# A tibble: 4 × 1
  Waterfront_yes
           <dbl>
1              0
2              0
3              1
4              0
```

Note: dummy encoding can be done with step_dummy() in a tidymodels recipe.

Multivariate Analysis — Building the Recipe

```
RecipeHouses=recipe(Price ~ .,
data=DataTrain) |>

step_dummy(Waterfront)
```

Here is how the recipe later on (in the workflow) transforms the data:

```
juice(RecipeHouses |> prep()) |>
head()
```

Sqft	Grade	Price	Waterfront_yes
1180	7	221900	0
770	6	180000	0
1200	7	189000	0
1250	7	230000	0
1070	7	252700	0
1220	7	240000	0

Multivariate Analysis — Building the Model Design

Unfitted Model:

```
ModelDesignHouses=linear_reg() |>
  set_engine("lm") |>
  set_mode("regression")
print(ModelDesignHouses)
```

```
Linear Regression Model Specification
(regression)

Computational engine: lm
```

Multivariate Analysis — Creating Workflow & Fitting to the Training Data

```
WFModelHouses = workflow() |>
      add_recipe(RecipeHouses) |>
      add_model(ModelDesignHouses)
|>
      fit(DataTrain)
tidy(WFModelHouses)
```

term	estimate	std.error	statistic	p.value
(Intercept)	-570056.1881	15133.004746	-37.66973	0
Sqft	179.5844	3.254633	55.17809	0
Grade	95214.1229	2547.528662	37.37509	0
Waterfront_yes	868338.1458	22199.734356	39.11480	0

```
glance(WFModelHouses)
```

r.squared	adj.r.squared	sigma	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.581415	0.581332	2385 74.3	7002.418	0	3	-208785.2	417580.4	417618.5	8.608231e+14	15124	15128

Multivariate Analysis — Predicting Testing Data and Metrics

```
DataTestWithPredictions =
augment(WFModelHouses,
new_data=DataTest)
metrics(DataTestWithPredictions,
truth=Price, estimate=.pred)
```

.metric	.estimator	.estimate
rmse	standard	2.446563e+05
rsq	standard	5.488493e-01
mae	standard	1.633577e+05

Exercise

Let’s Run the Analysis

Download: 05-LinRegrExerc100.Rmd